ENGENHARIA GRAMATICAL 2026/27 · PROJETO, PARTE I · CHECKPOINT 1

Sentinela
uma linguagem para playbooks de resposta a incidentes

Playbooks de segurança e de rede **verificados antes de serem ativados**, especificados por uma gramática de atributos em LarkAG.

Grupo 7 · Ivo Sousa · PG63976 · Miguel Páscoa · PG64007 · Ricardo Eusébio · PG64022

Mestrado em Engenharia Informática · Universidade do Minho

ROTEIRO

O que vamos mostrar, pela ordem das tarefas do guião

1

Desenho da linguagem

Problema, domínio, modelo de execução, exemplos e casos práticos.

2

Léxico e sintaxe

Tokens, gramática BNF P1–P55, LALR(1) sem conflitos.

3

Semântica

Gramática de atributos em LarkAG: atributos, regras, 32 erros e 6 avisos.

4

Qualidade

Fatores do NB07 e métricas da gramática e da GA.

E no fim: demonstração ao vivo, testes, limitações e o que fica para a Parte II.

Os playbooks em YAML só revelam os erros durante um incidente real

Playbook numa plataforma SOAR (YAML): **aceite sem nenhum aviso**

```yaml
- name: block
  action: block_ip
  input: "{{ alert.sorce_ip }}"        # ① campo mal escrito
- name: notify
  action: send_email
  body: "Chave: {{ secrets.api_key }}" # ② segredo por email
- name: isolate
  action: isolate_host                 # ③ sem condição
```

São 3 da manhã, um IP faz milhares de tentativas de SSH… e o bloqueio falha porque o campo não existe.

O mesmo playbook em **Sentinela**: rejeitado **antes** de ser ativado

`E204` ① o alerta não tem o campo `sorce_ip`

`E601` ② um segredo não pode ir para uma mensagem

`W01` ③ ação destrutiva sem nenhuma condição

> Os erros passam a ser apanhados **às 3 da tarde, quando se escreve o playbook**, e não às 3 da manhã, durante o ataque.

Uma DSL em que os conceitos de segurança e de rede são construções da linguagem

Utilizadores

Analistas de segurança (SOC) e engenheiros de redes e de operações.

Objetivo

Playbooks **legíveis** e **verificáveis**: um playbook com erros nunca chega a produção.

Domínio

Resposta automática a incidentes: port scan, brute force, flood, ARP spoofing.

CONCEITO	CONSTRUÇÃO	PARA QUE SERVE
Alerta	`alert nome { campo: tipo; }`	esquema tipado dos dados que chegam do IDS
Playbook	`playbook nome { … }`	conjunto de reações, com estado persistente
Handler	`on X as a { }` · `every 1h { }`	reage a cada alerta, ou à passagem do tempo
Tipos de rede	`ip` · `cidr` · `duration`	não se confunde um IP com uma rede: `10.0.0.0/8`, `24h`
Segredo	`secret token: string = …;`	credencial cujo fluxo é verificado

Um playbook é um serviço sempre ativo, verificado antes de ser ativado

playbook.snt
escrito pelo analista

→

Gramática de atributos
sintaxe + semântica

→

✘ Erro → rejeitado
o playbook não é ativado

✔ Avisos → ativo
os avisos ficam registados

`on X as a`

Corre sempre que chega um alerta do tipo `X`.

`every 1h`

Corre periodicamente, haja ou não alertas.

Estado persistente

As variáveis do playbook mantêm o valor entre execuções.

Estado local

As variáveis de um handler recomeçam em cada execução. Um handler de cada vez.

Brute force SSH: bloquear um IP externo com demasiadas tentativas

```
alert ssh_failed_login {
  src_ip: ip;
  user: string;
}

playbook brute_force_ssh {
  var limite: int = 5;
  var rede_interna: cidr = 10.0.0.0/8;

  on ssh_failed_login as a {
    var tentativas: int = count(a.src_ip, 10m);
    if (tentativas > limite and not (a.src_ip in rede_interna)) {
      block_ip(a.src_ip, 24h);
      notify("Brute force de " + str(a.src_ip));
    }
  }
}
```

Esquema do alerta

Os campos que chegam do IDS, com tipo. `a.src_ip` é verificado contra este esquema.

Tipos e literais de rede

`10.0.0.0/8` é um `cidr`, `24h` uma `duration`; `ip in cidr` é uma operação da linguagem.

Biblioteca de ações

`block_ip`, `quarantine`, `rate_limit`, `log`, `notify` · funções `count`, `len`, `str`.

Todas as construções exigidas pelo guião, ao serviço do domínio

REQUISITO DO GUIÃO	NA SENTINELA
Variáveis e declarações	`var nome: tipo = valor;` · `secret nome: tipo = valor;`
Scopes bem definidos	**Estáticos**: playbook ⊃ handler ⊃ bloco. Declarar antes de usar; sem redeclarar no mesmo scope; *shadowing* com aviso; variável do `for` e alias do alerta só de leitura.
Expressões e estado	`or and not` · `== != < > <= >= in` · `+ -` · `a.campo` · atribuição e estado persistente
Construções aninhadas	handlers com `if`/`else` e `for` aninhados sem limite
Ramificação e repetição	`if (…) { } else { }` · `for x in lista { }` · repetição no tempo com `every`
Semântica além da sintaxe	**32 erros e 6 avisos** verificados pela gramática de atributos

Tipos: `int bool string ip cidr duration list<T>`

Quatro ataques reais, do mais fácil ao mais difícil

`fácil`

Port scan

`nmap -sS`

Bloqueia quem varre portas sensíveis (22, 3389, 5432). `if/else`, `in` sobre listas.

`fácil`

Brute force SSH

`hydra`

Bloqueia IPs externos com demasiadas tentativas. `count`, `ip in cidr`.

`intermédio`

SYN flood

`hping3 --flood`

Limita cada origem fora da *whitelist*. `for` com `if` aninhado, contador.

`difícil`

ARP spoofing

`arpspoof`

Isola o atacante na LAN. `secret`, estado persistente, `every`, várias ações.

> Os quatro estão em `exemplos/` e passam a verificação **sem erros nem avisos**. Cobrem as camadas de aplicação, transporte e ligação (L2).

ARP spoofing: segredo, estado persistente e `every`

```
playbook lan_integrity {
  secret webhook_token: string = "xoxb-123";
  var lan: cidr = 10.0.2.0/24;
  var servidores: list<ip> = [10.0.1.10, 10.0.1.20];
  var incidentes: int = 0;          # estado persistente

  on arp_spoof as x {
    var critico: bool = x.claimed_ip == x.gateway_ip;
    for srv in servidores {
      if (x.claimed_ip == srv) { critico = true; }
    }
    if (critico and x.attacker_ip in lan) {
      quarantine(x.attacker_ip);
      block_ip(x.attacker_ip, 1h);
      incidentes = incidentes + 1;
      notify("ARP spoofing por " + str(x.attacker_ip), webhook_token);
    } else {
      log("ARP suspeito: " + str(x.attacker_ip));
    }
  }

  every 1h {                        # haja ou não alertas
    if (incidentes > 0) {
      notify("Resumo: " + str(incidentes), webhook_token);
      incidentes = 0;
    }
  }
}
```

Segredo protegido

O `webhook_token` só pode ir para o 2.º argumento do `notify` (credencial). Num `log` seria o erro `E601`

Estado entre alertas

`incidentes` acumula de ataque em ataque e o `every 1h` envia o resumo e repõe o contador.

Ações destrutivas guardadas

`quarantine` e `block_ip` estão dentro de um `if`: fora dele seria o aviso `W01`

COMO CHEGÁMOS AQUI

Da proposta inicial à v2.0: uma linguagem mais pequena e uma só especificação

v0 · proposta

6 casos, ações do utilizador, `retry`, interpolação `${…}`, `else if`, tipos `port` e `host`.

110 produções · professor: "pode ficar demasiado grande"

v1 · núcleo

4 casos; cortes no que não era essencial; **32 erros** definidos; playbook sempre ativo com `every`.

gramática Lark + verificador Python

v1.3 · qualidade

Análise do NB07: nomes consistentes, métricas, GA completa especificada; um exemplo em LarkAG.

avaliação de qualidade documentada

v2.0 · LarkAG

Toda a linguagem num só ficheiro LarkAG; novo aviso **W06**; testes refeitos.

65 produções · **112** testes

−41%

produções (110 → 65)

1

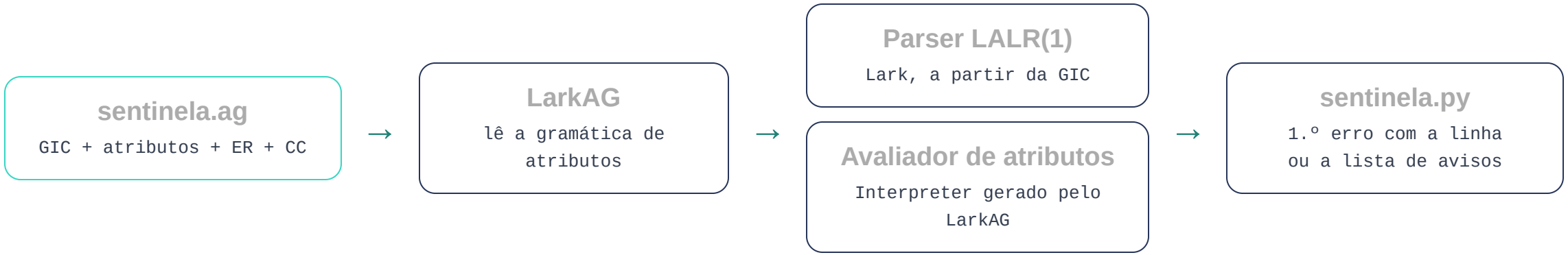
ficheiro define tudo: `sentinela.ag`

38

verificações (32 erros + 6 avisos)

ARQUITETURA

Uma só fonte de verdade: o parser e o avaliador são gerados a partir de `sentinela.ag`



1 · Sintaxe

O texto é lido pelo parser LALR(1) gerado da GIC. Programa mal formado → erro de sintaxe com a linha.

2 · Semântica

O avaliador calcula os atributos numa travessia e verifica as condições de contexto. A primeira que falha **para** a avaliação.

3 · Resultado

Erro → playbook **rejeitado**. Sem erros → **ativo**, com os avisos do atributo `warnings` da raiz.

Os literais de rede são tokens próprios, com prioridades no lexer

TOKEN	DEFINIÇÃO	EXEMPLO
`NAME`	`[a-zA-Z_][a-zA-Z0-9_]*`	`src_ip`
`INT`	`\d+`	`5432`
`STRING`	`"[^"]*"`	`"bloqueado"`
`IP` prio 4	`\d{1,3}(\.\d{1,3}){3}`	`10.0.0.5`
`CIDR` prio 5	`…/\d{1,2}`	`10.0.0.0/8`
`DURATION` prio 3	`\d+(s|m|h)`	`24h`
`BOOL` · `RELOP`	`true false` · `== != < > <= >=`	
Comentários	`#` até ao fim da linha	

21 palavras reservadas

`alert playbook on as every var secret if else for in not and or list int bool string ip cidr duration` (+ `true`, `false`)

Prioridades

`10.0.0.0/8` é **um** `CIDR`, não um `IP` seguido de outra coisa; `24h` é **uma** `DURATION`, não `INT` + `NAME`.

Léxico vs semântica

O lexer aceita `300.1.1.1`; é a semântica que o rejeita (`E701`).

Uma só gramática BNF, LALR(1) sem conflitos

```
P1   start      → alerts playbook
P10  playbook   → "playbook" NAME "{" statedecls handlers "}"
P15  handler    → "on" ident "as" ident block
P16  handler    → "every" DURATION block
P19  stmts      → stmts stmt      # recursão à esq.
P21  vardecl    → "var" NAME ":" datatype "=" expr ";"
P25  stmt       → "if" "(" expr ")" block
P27  stmt       → "for" NAME "in" expr block
P28  stmt       → callee "(" args ")" ";"
P33  expr       → expr "or" andexpr
P40  relexpr    → addexpr "in" addexpr
P45  accessexpr → accessexpr "." NAME
P47  primary    → NAME
… 55 produções numeradas (GRAMATICA.md §2)
```

`expr · or`

`andexpr · and`

`notexpr · not`

`relexpr · == != < > <= >= in`

`addexpr · + -`

`accessexpr · .campo`

`primary · nomes, literais, ( ), [ ]`

Precedência por estratificação: quanto mais abaixo, mais forte liga.

55	0	25	132
produções	conflitos LALR	não-terminais	estados LALR

Sem ambiguidade (LALR(1)) · associatividade à esquerda · comparações não associativas · sem *dangling else* (blocos sempre com `{ }`)

Primeiro, os domínios em que os atributos tomam valores

DOMÍNIO	DEFINIÇÃO	LEITURA
`Tipo`	`{int, bool, string, ip, cidr, duration}` ∪ `list<T>` ∪ `A:n`	`A:n` é o tipo do alias de um alerta `n`
`Espécie`	`{var, secret, loop, alias}`	o que um nome é (variável, segredo, variável do `for`, alias)
`Env`	`Nome ⇀ Tipo × Espécie`	ambiente: os nomes visíveis
`Scope`	`℘(Nome)`	nomes declarados no scope atual
`AlertTab`	`Nome ⇀ (Nome ⇀ Tipo)`	todos os alertas e os seus campos
`Sig`	`(params, min, returns, destructive)`	assinatura de uma ação da biblioteca
`Level`	`{top, inner}`	declaração no estado do playbook ou num bloco
`Warning`	`(código, linha, mensagem)`	um aviso

Para cada símbolo: que atributos tem, herdados ↓ ou sintetizados ↑, e o domínio

```
A(playbook)   = { (alerttab, ↓, AlertTab),
                  (handled,  ↑, ℘(Nome)), (warnings, ↑, [Warning]) }
A(handler)    = { (env, ↓, Env), (alerttab, ↓, AlertTab),
                  (seen, ↓, ℘(Nome)),
                  (target, ↑, Nome), (used, ↑, ℘(Nome)),
                  (warnings, ↑, [Warning]) }
A(stmt)       = { (env, ↓, Env), (scope, ↓, Scope),
                  (alerttab, ↓, AlertTab), (guarded, ↓, 𝔹),
                  (envout, ↑, Env), (scopeout, ↑, Scope),
                  (declared, ↑, Nome ⇀ ℕ), (used, ↑, ℘(Nome)),
                  (warnings, ↑, [Warning]) }
A(vardecl)    = A(stmt) sem guarded, com (level, ↓, Level)
A(callee)     = { (name, ↑, Nome), (signature, ↑, Sig ∪ {⊥}),
                  (line, ↑, ℕ) }
A(X)          = { (env, ↓, Env), (alerttab, ↓, AlertTab),
                  (type, ↑, Tipo), (secret, ↑, 𝔹),
                  (const, ↑, Const), (used, ↑, ℘(Nome)) }
  para X ∈ {expr, andexpr, notexpr, relexpr, addexpr, accessexpr, primary}
```

Herdados ↓

vêm do pai ou dos irmãos à esquerda: o **contexto** (`env`, `scope`, `alerttab`, `guarded`, `level`, `seen`)

Sintetizados ↑

calculados a partir dos filhos: os **resultados** (`type`, `secret`, `envout`, `used`, `warnings`)

No LarkAG

São as linhas `IA(...)` e `SA(...)` no início de `sentinela.ag`, com os tipos Python de cada domínio.

Declaração completa: `GRAMATICA.md` §3.2 · 25 nomes de atributos

Só com as declarações dos atributos, já se leem as regras da linguagem

PADRÃO NAS DECLARAÇÕES	REGRA DA LINGUAGEM
`env` ↓ em todo o lado; `envout` ↑ nas sequências	**Scopes estáticos** e **declarar antes de usar**: o ambiente desce e cresce da esquerda para a direita
`scope` ↓, um por bloco	Não se redeclara no mesmo bloco; um bloco interior pode reutilizar um nome
`Env` guarda a `Espécie`	Segredos, variáveis do `for` e aliases têm **regras próprias**
`type` ↑ nas expressões, sem tipo herdado	**Tipagem de baixo para cima**; os tipos das variáveis vêm de declarações explícitas
`secret` ↑ nas expressões	**Fluxo de informação**: o que usa um segredo fica secreto
`seen` ↓ em `alert`, `field`, `handler`	Cada alerta, campo e `on` é **único**
`level` ↓ em `vardecl` · `guarded` ↓ nas instruções	A **posição** importa: segredos só no estado; ações destrutivas fora de `if` assinaladas
`warnings` é atributo; os erros são CC	**Dois níveis**: erros impedem a ativação, avisos não

Cada produção tem regras de cálculo (ER) e condições (CC)

Notação matemática (GRAMATICA.md §4)

P21 `vardecl → "var" NAME ":" datatype "=" expr ";"`

ER

$$vardecl.envout = vardecl.env \oplus \{NAME \mapsto (datatype.type,\ var)\}$$

$$vardecl.scopeout = vardecl.scope \cup \{NAME\}$$

CC

$$NAME \notin vardecl.scope \rightarrow E102$$

$$expr.type = datatype.type \rightarrow E301$$

$$\neg\ expr.secret \rightarrow E602$$

P47 `primary → NAME`

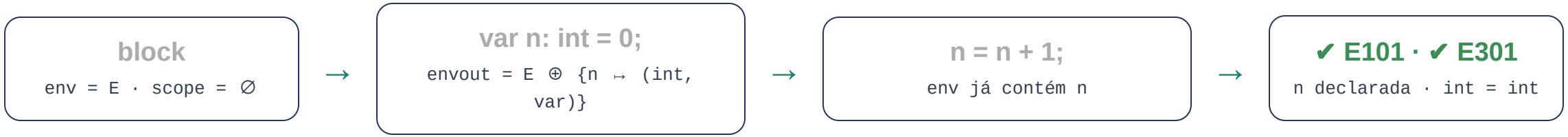
$$(primary.type,\ k) = primary.env(NAME) \cdot primary.secret = (k = secret)$$

$$NAME \in dom(primary.env) \rightarrow E101$$

A mesma produção em LarkAG (`sentinela.ag`)

```
vardecl : "var" NAME ":" datatype "=" expr ";"
ER {
  expr.env = vardecl.env;
  expr.alerttab = vardecl.alerttab;
  vardecl.envout = vardecl.env |
      { str(NAME) : (datatype.type, "var") };
  vardecl.scopeout = vardecl.scope | { str(NAME) };
  …
}
CC {
  str(NAME) not in vardecl.scope
      -> "E102: nome já declarado neste scope";
  expr.type == datatype.type
      -> "E301: o valor não tem o tipo declarado";
  not expr.secret
      -> "E602: um segredo não pode ser guardado…";
}
```

A gramática é L-atribuída: uma só travessia, da esquerda para a direita



Porque é L-atribuída

Cada atributo herdado depende só do **pai** e dos **irmãos à esquerda**:

- `handlers.env = statedecls.envout` (P10)
- `stmt.env = stmts₁.envout` (P19)
- `handler.seen = handlers₁.handled` (P13)
- tipo da variável do `for` = `elem(expr.type)` (P27)

Consequências

- uma travessia em profundidade chega para tudo;
- o LarkAG calcula a ordem a partir das dependências e recusa ciclos;
- a **primeira** condição de contexto que falha para a avaliação, como num compilador, e a linha do erro é a do nó em avaliação.

32 erros e 6 avisos, organizados em 7 famílias

5
Declarações e scopes
E101–E105 · não declarado, redeclarado, alerta/campo repetido, segredo fora do estado

4
Alertas e handlers
E201–E204 · alerta inexistente, `on` repetido, campo inexistente

10
Tipos
E301–E310 · `ip ≠ cidr`, `"porta" + 22`, `if (s.port)`, `[]`

5
Chamadas
E401–E405 · ação inexistente, argumentos, `log();`

2
Mutabilidade
E501–E502 · alterar um segredo ou a variável do `for`

3
Segredos
E601–E603 · segredo num `log`, numa `var` ou numa condição

3
Literais de rede
E701–E703 · `300.1.1.1`, `/40`, `0h`

6
Avisos
W01–W06 · destrutiva sem `if`, não usada, *shadowing*, alerta sem `on`, `if (true)`, `log("")`

Erros = condições de contexto (o playbook não é ativado) · Avisos = atributo `warnings` (o playbook é ativado) · Catálogo completo: `CATALOGO_ERROS.md`

Os segredos são seguidos ao longo das expressões, até onde são usados

✔ Aceite

```
secret token: string = "xoxb-123";
# o segredo vai só para a credencial do notify
notify("Scan de " + str(s.src), token);
```

✘ Rejeitado

```
log("token=" + token);          # E601
var copia: string = token;      # E602
if (token == "abc") { … }       # E603
token = "outro";                # E501
```

Atributo `secret` ↑

É verdadeiro se a expressão usa um segredo: `"x" + token`, `str(token)`, `token == "abc"`.

Fuga explícita

Um valor secreto não pode ir para nenhuma ação, nem ser guardado numa `var` (para não ser "lavado").

Fuga implícita

Também não pode controlar um `if` ou um `for`: o comportamento revelaria o segredo.

DEMONSTRAÇÃO

O processador rejeita no primeiro erro e ativa o playbook mesmo com avisos

```
$ python sentinela.py testes/E204*.snt testes/E301*.snt testes/E601*.snt \
                      testes/W02*.snt exemplos/04_arp_spoofing.snt
✘ rejeitado  testes/E204_campo_inexistente.snt
    linha 9: erro E204: o alerta não tem esse campo
✘ rejeitado  testes/E301_tipo_declarado.snt
    linha 8: erro E301: o valor não tem o tipo declarado
✘ rejeitado  testes/E601_segredo_no_log.snt
    linha 10: erro E601: um segredo só pode ser passado como credencial do notify
✔ ativo  testes/W02_variavel_nao_usada.snt
    linha 9: aviso W02: 'esquecida' é declarada mas nunca é usada
✔ ativo  exemplos/04_arp_spoofing.snt
```

`python sentinela.py exemplos/*.snt`

os 4 casos: ativos, sem avisos

`python sentinela.py testes/*.snt`

um programa por código: cada erro e aviso

`python sentinela.py --arvore …`

mostra também a árvore sintática

VALIDAÇÃO

112 testes automáticos: cada código tem um programa que o provoca na linha certa

```
# E302: Somar valores de tipos diferentes.
alert scan {
  src: ip;
  port: int;
}

playbook p {
  on scan as s {
    log("porta " + s.port);   # expect: E302
  }
}
```

O teste verifica o **código** e a **linha**. Para avisos, verifica que o playbook fica ativo com exatamente esse aviso.

112

testes a passar (`pytest -q`)

Sintaxe

LALR(1) sem conflitos · aceita todos os programas · rejeita mal formados · tokens de rede.

Semântica

41 programas em `testes/` (todos os 38 códigos) · exemplos sem avisos · casos válidos que **não** podem ser rejeitados.

A Sentinela troca extensibilidade por fiabilidade, unicidade e consistência

FATOR	NÍVEL	PORQUÊ
Fiabilidade	`alta`	É o objetivo: tipos estáticos, campos validados, fluxo de segredos, ações destrutivas assinaladas
Expressividade	`alta`	Tipos, literais e ações de rede são primitivos: escreve-se ao nível do problema
Unicidade	`alta`	Uma só forma de cada construção: um ciclo, um condicional, uma declaração
Consistência	`alta`	`nome: tipo` em todas as declarações, `;` e `{ }` uniformes, uma só convenção de nomes
Documentação	`média`	Só comentários de linha; os esquemas `alert` documentam os dados de forma verificável
Escalabilidade	`média`	Vários alertas e estado partilhado, mas sem módulos
Extensibilidade · Modularidade	`baixa`	**Por opção**: sem ações do utilizador nem `import`, para manter a linguagem pequena

A GA cresce sobretudo com regras de cópia: a lógica real é pequena

Gramática independente de contexto

Não-terminais · terminais	**25 · 46**
Produções (Lark)	**65**
Produções unitárias	**10** (6 na escada de precedência)
Recursão direta (esq. / dir.)	**11 / 1** (só o prefixo `not`)
Estados LALR · entradas	**132 · 849**

Gramática de atributos

Atributos (↓ / ↑)	**123** (44 / 79) · 25 nomes
Regras de cálculo	**354**
das quais cópias `X.a = Y.a`	**190 (54 %)**
Condições de contexto	**54** · 32 códigos
Grau médio de dependências	**1,81**

54%

das regras só copiam contexto (obrigatório no LarkAG)

164

regras com lógica real, mais 54 condições

110 → 65

produções, da proposta à v2.0

DECISÕES E LIMITES

O que decidimos deixar de fora, e o que nenhuma análise estática apanha

Fora do âmbito, de propósito

- ações e funções definidas pelo utilizador, módulos e `import`;
- `retry … until`, `else if` em cadeia, `*` e `/`;
- interpolação de strings `${…}` (usa-se `+` e `str()`);
- tipos `port` e `host` (usam-se `int` e `ip`).

Motivo: manter a gramática pequena e o projeto realizável.

Só se sabe em execução

- o alerta real (JSON do IDS) chega sem um campo declarado;
- a firewall falha ao aplicar o `block_ip`;
- `count(…)` devolve um valor inesperado;
- `log(x + "")`: a mensagem vazia não é detetável estaticamente.

Por isso o W06 só deteta `""` escrito diretamente.

> E uma escolha de comportamento: **a primeira condição de contexto que falha para a avaliação**, como no LarkAG das aulas e num compilador.

PRÓXIMOS PASSOS · PARTE II

A árvore anotada pela gramática de atributos é a base da Parte II

① **Representar**

Grafo de fluxo de controlo de cada handler.

② **Analisar**

Código inalcançável, ações destrutivas alcançáveis sem condição.

③ **Visualizar**

Desenho dos playbooks e dos grafos.

④ **Transformar**

Traduzir playbooks em regras `nftables` reais.

Os atributos já calculados (tipos, scopes, segredos, `guarded`) são o ponto de partida para estas análises.

CONCLUSÃO

A Sentinela deteta **antes da ativação** os erros que o YAML só revela em produção

1
especificação: `sentinela.ag`

0
conflitos LALR(1)

38
verificações semânticas

112
testes a passar

Obrigado! Perguntas?

Repositório: `github.com/icsousa/EG` · pasta `TP`

Referências

- N. Macedo, "NB07: Language and Grammar Quality", Engenharia Gramatical, Universidade do Minho, 2026.
- N. Macedo, "EX04: Attribute Grammars and LarkAG", Engenharia Gramatical, Universidade do Minho, 2026.
- P. R. Henriques, "Brincando às Linguagens com rigor: Engenharia Gramatical", Universidade do Minho, 2012.
- Lark, a parsing toolkit for Python: `github.com/lark-parser/lark`
- LarkAG, attribute grammars on top of Lark: `github.com/nmacedo/LarkAG`
- Sentinela (este projeto): `github.com/icsousa/EG`, pasta `TP` · `README.md`, `GRAMATICA.md`, `CATALOGO_ERROS.md`, `QUALIDADE.md`